Soft Nexis Technology

Virtual Internship Programme · Data Science & ML in Python

PROJECT 2

Machine Learning Classification with Scikit-Learn

Data Science & ML in Python – Virtual Internship
Soft Nexis Technology

Duration	4 Weeks
Difficulty	Beginner – Intermediate
Tasks	2 (Decision Tree | Random Forest + Evaluation)
Dataset	Iris Flower Dataset (classic ML benchmark)
Language	Python 3.x | Jupyter Notebook

Build your first Machine Learning model from scratch! Every step is explained in plain language with complete runnable Python code. By the end you will have trained, evaluated, and improved a real ML classifier.

Project Overview

Machine Learning Classification means training a computer to automatically assign a category label to new input data based on examples it has seen. Examples you use every day: Gmail classifying emails as Spam or Not Spam; Banks approving or rejecting loan applications; Doctors' apps classifying an X-ray as Normal or Abnormal.

The core idea: *You show the computer many examples (training data) with known answers (labels). The computer finds patterns. Then it uses those patterns to predict the label for new, unseen data.*

Project Tasks

- **Task 1 –** Build a Decision Tree Classifier and understand how it makes decisions
- **Task 2 –** Build a Random Forest, compare models, and evaluate with proper metrics

Key ML Vocabulary (Learn These!)

Term	Plain English Explanation
Feature	An input variable (e.g., petal length, sepal width). Also called X.
Label / Target	The answer we want to predict (e.g., flower species). Also called y.
Training Set	Data used to teach the model (usually 70-80% of total data).
Test Set	Data held back to evaluate the model (20-30%). Model never sees this during training.
Accuracy	% of predictions that are correct. E.g., 95% accuracy = 95 out of 100 correct.
Overfitting	Model memorises training data but fails on new data. Like memorising answers without understanding.

Environment Setup

1 Install Required Libraries

Open your terminal and run:

```
pip install scikit-learn pandas matplotlib seaborn jupyter
```

scikit-learn is the main ML library. It contains 30+ algorithms ready to use with just a few lines of code.

2 About the Iris Dataset

The **Iris dataset** is the 'Hello World' of Machine Learning. It contains 150 iris flowers from 3 species: Setosa, Versicolor, Virginica. For each flower, 4 measurements are recorded: sepal length, sepal

width, petal length, petal width (all in cm). Your ML model must predict which species a flower belongs to based on these 4 measurements.

```
from sklearn.datasets import load_iris
import pandas as pd

# Load the built-in Iris dataset
iris = load_iris()

# Convert to DataFrame for easy viewing
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

print('Dataset shape:', df.shape)
print('Species:', df['species'].unique())
print(df.head())
```

Output → Dataset shape: (150, 5) Species: ['setosa' 'versicolor' 'virginica']

TASK 1 Decision Tree Classifier – Build Your First ML Model

Task Description

A **Decision Tree** is the most intuitive ML algorithm – it makes decisions exactly like a flowchart. At each node, it asks a yes/no question about a feature (e.g., 'Is petal length < 2.45 cm?'). Based on the answer, it goes left or right until it reaches a leaf node with the predicted class.

Everyday analogy: *You are playing 20 Questions to guess an animal. You ask: 'Does it have 4 legs? Does it bark? Is it bigger than a cat?' Each question splits possibilities further. A Decision Tree does the same thing automatically, learning which questions to ask from data!*

Library	Purpose
sklearn.tree.DecisionTreeClassifier	The Decision Tree model
sklearn.model_selection.train_test_split	Split data into train/test sets
sklearn.metrics.accuracy_score	Calculate accuracy of predictions
matplotlib.pyplot	Plot the decision tree visually

Step-by-Step Guide

1 Load and Prepare Data

We need to separate Features (X) from Labels (y) before training:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

# Load dataset
iris = load_iris()

# X = Features (the 4 measurements)
X = iris.data
print('Features shape:', X.shape)   # (150 rows, 4 columns)

# y = Labels (0=setosa, 1=versicolor, 2=virginica)
y = iris.target
print('Labels shape:', y.shape)     # (150,)
print('Unique classes:', set(y))    # {0, 1, 2}
```

Output → Features shape: (150, 4) Labels shape: (150,) Unique classes: {0, 1, 2}

2 Split Data: Training Set vs Test Set

This is the most important step in ML! We split data so the model is trained on one portion and evaluated on a completely separate portion it has never seen. This tests real-world performance.

```
# Split 80% training, 20% testing
# random_state=42 makes split reproducible (same result every run)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,     # 20% for testing = 30 samples
    random_state=42,    # Seed for reproducibility
    stratify=y          # Keep equal proportion of each class
)

print(f'Training samples: {X_train.shape[0]}')   # 120
print(f'Testing  samples: {X_test.shape[0]}')    # 30
print(f'Training class distribution: {dict(zip(*np.unique(y_train, return_counts=True)))}'
)
```

stratify=y *ensures all 3 classes are equally represented in both train and test sets. Without this, you might accidentally put all Setosa in training and test only on Versicolor/Virginica.*

3 Train the Decision Tree Model

Training is just one line of code! Scikit-learn handles all the math:

```
from sklearn.tree import DecisionTreeClassifier

# Create the model
# max_depth=3 limits tree to 3 levels deep (prevents overfitting)
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)

# TRAIN the model on training data
# .fit() is the actual learning step
dt_model.fit(X_train, y_train)

print('Model training complete!')
print('Number of leaves in tree:', dt_model.get_n_leaves())
print('Tree depth:', dt_model.get_depth())
```

Output → `Model training complete! Number of leaves: 6 Tree depth: 3`

fit(X_train, y_train) is the learning step. The model looks at 120 examples and figures out the best questions to ask. This takes milliseconds for small datasets!

4 Make Predictions

Now use the trained model to predict on data it has never seen:

```
# Predict on the test set
y_pred = dt_model.predict(X_test)

print('Actual labels:   ', y_test[:10])
print('Predicted labels:', y_pred[:10])

# Predict a SINGLE new flower
# A flower with: sepal_l=5.1, sepal_w=3.5, petal_l=1.4, petal_w=0.2
new_flower = [[5.1, 3.5, 1.4, 0.2]]
```

```
prediction = dt_model.predict(new_flower)
prediction_proba = dt_model.predict_proba(new_flower)

print('\nPredicted species:', iris.target_names[prediction[0]])
print('Confidence:', prediction_proba)
```

Output → Predicted species: setosa Confidence: [[1.0, 0.0, 0.0]]

5 Evaluate the Model

Accuracy alone doesn't tell the full story. Use these metrics:

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Accuracy: overall % correct
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy*100:.2f}%')

# Classification Report: detailed per-class metrics
print('\nClassification Report:')
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names, ax=ax)
ax.set_title('Decision Tree Confusion Matrix', fontsize=13, fontweight='bold')
ax.set_xlabel('Predicted', fontsize=11)
ax.set_ylabel('Actual', fontsize=11)
plt.tight_layout()
plt.savefig('confusion_matrix_dt.png', dpi=150)
plt.show()
```

Reading the Confusion Matrix: *Each row = actual class, each column = predicted class. Diagonal = correct predictions. Off-diagonal = mistakes. Ideally all numbers should be on the diagonal.*

6 Visualise the Decision Tree

One of the biggest advantages of Decision Trees is that you can **see exactly how it makes every decision**:

```
from sklearn.tree import plot_tree

fig, ax = plt.subplots(figsize=(14, 7))
plot_tree(dt_model,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,          # Colour nodes by predicted class
          rounded=True,         # Rounded corners
          fontsize=10,
          ax=ax)
ax.set_title('Decision Tree – Iris Classifier\n(Soft Nexis Technology Internship)',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('decision_tree_visual.png', dpi=150, bbox_inches='tight')
plt.show()
print('Decision tree saved!')
```

How to read the tree: At the top (root) node, you see the first question (e.g., petal length <= 2.45). True = go left, False = go right. Colour shows which class dominates that node.

7 Experiment with Tree Depth

max_depth controls how complex the tree grows. Let's test different depths:

```
import numpy as np

depths = [1, 2, 3, 4, 5, 10, None]  # None = unlimited depth
train_acc = []
test_acc  = []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_acc.append(model.score(X_train, y_train))
    test_acc.append(model.score(X_test, y_test))

# Plot
fig, ax = plt.subplots(figsize=(9, 5))
depth_labels = [str(d) for d in depths]
ax.plot(depth_labels, train_acc, 'o-', label='Training Accuracy', color='#7B1FA2', linewid
th=2)
ax.plot(depth_labels, test_acc, 's--', label='Test Accuracy', color='#E65100', linewidth=2
)
ax.set_title('Training vs Test Accuracy by Tree Depth', fontsize=13, fontweight='bold')
ax.set_xlabel('max_depth (None = unlimited)', fontsize=11)
ax.set_ylabel('Accuracy', fontsize=11)
ax.legend(fontsize=10)
ax.set_ylim(0.8, 1.02)
plt.tight_layout()
plt.savefig('depth_vs_accuracy.png', dpi=150)
plt.show()
```

Key observation: As depth increases, training accuracy goes to 100% but test accuracy may drop – that is overfitting. The model memorised training data instead of learning general patterns.

TASK 2 Random Forest – A Smarter Model + Full Evaluation

Task Description

A **Random Forest** is a collection of many Decision Trees working together – like taking a vote from 100 experts instead of asking just one. Each tree is trained on a slightly different random subset of data and features. The final prediction is the majority vote across all trees. This dramatically reduces overfitting.

> ***Analogy:*** *Imagine predicting tomorrow's weather. Would you trust one meteorologist or the average prediction of 100 meteorologists? The wisdom of the crowd is almost always more reliable. Random Forest applies this idea to ML!*

Library	Purpose
sklearn.ensemble.RandomForestClassifier	Random Forest model (ensemble of trees)
sklearn.model_selection.cross_val_score	Cross-validation for reliable accuracy
sklearn.metrics.roc_auc_score	ROC-AUC score for model quality
sklearn.preprocessing.StandardScaler	Feature scaling (normalize values)

Step-by-Step Guide

1 Train the Random Forest

The API is identical to Decision Tree – just swap the model class:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.20, random_state=42, stratify=y)

# Create Random Forest with 100 trees
rf_model = RandomForestClassifier(
    n_estimators=100,   # Number of trees (100 is a good start)
    max_depth=5,        # Max depth per tree
    random_state=42,
    n_jobs=-1           # Use all CPU cores for faster training
)
```

```
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print('Random Forest Accuracy:', accuracy_score(y_test, y_pred_rf)*100, '%')
```

Output → Random Forest Accuracy: 100.0 % (may vary slightly with different seeds)

2 Cross-Validation – A More Reliable Accuracy Estimate

A single train/test split might be lucky. **Cross-validation (CV)** splits the data into 5 folds, trains 5 times (each time holding out a different fold as test set), and averages the 5 accuracy scores. This gives a much more honest picture of performance.

```
from sklearn.model_selection import cross_val_score
import numpy as np

# 5-fold cross-validation on Decision Tree
dt_cv = cross_val_score(DecisionTreeClassifier(max_depth=3, random_state=42),
                        X, y, cv=5, scoring='accuracy')

# 5-fold cross-validation on Random Forest
rf_cv = cross_val_score(RandomForestClassifier(n_estimators=100, random_state=42),
                        X, y, cv=5, scoring='accuracy')

print('Decision Tree CV Accuracy:')
print(f'  Each fold: {dt_cv.round(3)}')
print(f'  Mean: {dt_cv.mean():.3f} | Std: {dt_cv.std():.3f}')

print('Random Forest CV Accuracy:')
print(f'  Each fold: {rf_cv.round(3)}')
print(f'  Mean: {rf_cv.mean():.3f} | Std: {rf_cv.std():.3f}')
```

Lower Std (standard deviation) = more stable model. A model with 92% ± 1% is better than one with 92% ± 10% because it performs consistently across different data subsets.

3 Feature Importance – Which Features Matter Most?

Random Forest tells you how much each feature contributed to predictions. This is extremely valuable in real projects!

```
import matplotlib.pyplot as plt
import numpy as np

# Get feature importances
importances = rf_model.feature_importances_
feature_names = iris.feature_names

# Sort by importance
sorted_idx = np.argsort(importances)[::-1]

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar(range(4), importances[sorted_idx],
```

```
            color=['#7B1FA2','#AB47BC','#CE93D8','#E1BEE7'],
            edgecolor='black', linewidth=0.8)

ax.set_xticks(range(4))
ax.set_xticklabels([feature_names[i] for i in sorted_idx], rotation=15, fontsize=11)
ax.set_title('Feature Importance – Random Forest (Iris)', fontsize=13, fontweight='bold')
ax.set_ylabel('Importance Score', fontsize=11)

# Add value labels on bars
for bar, imp in zip(bars, importances[sorted_idx]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{imp:.3f}', ha='center', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150)
plt.show()
```

Expected result: *Petal length and petal width will have the highest importance (together ~90%). Sepal features are less useful for distinguishing the three Iris species.*

4 Compare Multiple Models Side by Side

Professional data scientists always compare several algorithms before choosing one:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

# Define models to compare
models = {
    'Decision Tree (depth=3)': DecisionTreeClassifier(max_depth=3, random_state=42),
    'Random Forest (100 trees)': RandomForestClassifier(n_estimators=100, random_state=42)
,
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5),
    'Logistic Regression': LogisticRegression(max_iter=200, random_state=42),
}

# Evaluate all models
results = {}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    results[name] = scores
    print(f'{name}: {scores.mean():.3f} ± {scores.std():.3f}')

# Visualise comparison
fig, ax = plt.subplots(figsize=(11, 6))
names = list(results.keys())
means = [results[n].mean() for n in names]
stds  = [results[n].std()  for n in names]

colors_bar = ['#E65100','#7B1FA2','#1B5E20','#01579B','#F57F17']
bars = ax.barh(names, means, xerr=stds, color=colors_bar,
               edgecolor='black', linewidth=0.7, height=0.6,
               error_kw={'capsize':5, 'linewidth':2})

for bar, mean in zip(bars, means):
    ax.text(mean + 0.002, bar.get_y() + bar.get_height()/2,
            f'{mean:.3f}', va='center', fontsize=10, fontweight='bold')

ax.set_xlabel('Cross-Validation Accuracy (mean ± std)', fontsize=11)
ax.set_title('Model Comparison – Iris Dataset\n(Soft Nexis Technology Internship)',
             fontsize=13, fontweight='bold')
ax.set_xlim(0.85, 1.05)
plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150)
plt.show()
```

5 Hyperparameter Tuning with GridSearchCV

Every ML model has settings called **hyperparameters** (e.g., n_estimators, max_depth). GridSearchCV automatically tries every combination and finds the best:

```
from sklearn.model_selection import GridSearchCV

# Define the grid of hyperparameters to try
param_grid = {
    'n_estimators': [50, 100, 200],     # Try 3 different tree counts
    'max_depth':    [3, 5, 10, None],   # Try 4 different depths
    'min_samples_split': [2, 5, 10],    # Minimum samples to split a node
}

# GridSearchCV tries ALL combinations (3x4x3 = 36 combos)
# with 5-fold CV each = 36 x 5 = 180 model fits!
grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,    # Use all CPU cores
    verbose=1
)

grid_search.fit(X_train, y_train)

print('Best Parameters:', grid_search.best_params_)
print('Best CV Accuracy:', grid_search.best_score_)

# Use best model for final predictions
best_model = grid_search.best_estimator_
final_accuracy = best_model.score(X_test, y_test)
print(f'Final Test Accuracy with best model: {final_accuracy*100:.2f}%')
```

Output → Best Params: {max_depth: None, min_samples_split: 2, n_estimators: 100} Best CV Accuracy: ~97%

6 Generate Full Evaluation Report

Combine everything into a professional evaluation summary:

```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

# Final predictions with best model
y_final_pred = best_model.predict(X_test)

print('='*55)
print('  FINAL MODEL EVALUATION REPORT')
print('  Soft Nexis Technology – ML Internship')
print('='*55)
print(f'  Algorithm  : Random Forest (Tuned)')
print(f'  Test Accuracy: {accuracy_score(y_test, y_final_pred)*100:.2f}%')
print()
print(classification_report(y_test, y_final_pred,
```

```
      target_names=iris.target_names))

# Final confusion matrix
fig, ax = plt.subplots(figsize=(7,5))
cm = confusion_matrix(y_test, y_final_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='BuPu',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names, ax=ax)
ax.set_title('Final Random Forest – Confusion Matrix', fontsize=13, fontweight='bold')
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual')
plt.tight_layout()
plt.savefig('final_confusion_matrix.png', dpi=150)
plt.show()
```

Precision: *Of all flowers predicted as Setosa, what % were actually Setosa?*
Recall: *Of all actual Setosa flowers, what % did the model find?*
F1-score: *Harmonic mean of Precision and Recall (balanced metric).*

Troubleshooting Guide

Problem	Solution
ImportError: No module named 'sklearn'	Run: pip install scikit-learn. If using Jupyter, restart kernel after installation.
Model accuracy is 0% or very low	Check that y_train and y_test use integer labels (0,1,2). Ensure X and y have matching lengths.
GridSearchCV takes too long	Reduce param_grid size. Use n_jobs=-1 to parallelize. Try fewer CV folds (cv=3).
ConvergenceWarning in LogisticRegression	Increase max_iter: LogisticRegression(max_iter=1000). Or add solver='lbfgs'.
Confusion matrix labels wrong order	Always pass target_names explicitly to classification_report() and as xticklabels/yticklabels.
cross_val_score gives different results each run	Add random_state=42 to your model AND use shuffle=True in KFold if creating manually.

Submission Checklist

- ■ Task 1: Jupyter Notebook with Decision Tree – all cells executed
- ■ Task 1: decision_tree_visual.png (tree diagram image)
- ■ Task 1: confusion_matrix_dt.png
- ■ Task 1: depth_vs_accuracy.png showing overfitting behaviour
- ■ Task 2: model_comparison.png (all 5 models compared)
- ■ Task 2: feature_importance.png (bar chart)
- ■ Task 2: final_confusion_matrix.png
- ■ Written report (250-350 words): Explain what overfitting is in your own words and how Random Forest solves it

Submit everything in a ZIP under **Project 2 – ML Classification**. Name your notebook: **YourName_Project2_Classification.ipynb**. Congratulations – you have built and evaluated real Machine Learning models!